

Law Smells

Defining and Detecting Problematic Patterns in Legal Drafting

Corinna Coupette

Max Planck Institute for Informatics
Saarbrücken, Germany

Dirk Hartung

Center for Legal Technology and Data
Science, Bucerius Law School
Hamburg, Germany

Janis Beckedorf

Ruprecht-Karls-Universität
Heidelberg, Germany

Maximilian Böther

Hasso Plattner Institute,
University of Potsdam
Potsdam, Germany

Daniel Martin Katz

Illinois Institute of Technology,
Chicago Kent College of Law
Chicago, Illinois, United States

Abstract

While some legal rules are formulated precisely enough to be implemented as programs, much of the law today is far from being computable. Given the similarities between law and software, however, even if legal rules are not currently computational in nature, we can leverage principles from computer science to assess their quality and pinpoint potential improvements. Therefore, adapting the idea of *code smells* from the software engineering subfield of refactoring, we initiate the study of *law smells*, i.e., patterns in legal texts that pose threats to the comprehensibility and maintainability of the law. With five intuitive law smells as running examples—namely, duplicated phrase, long element, large reference tree, ambiguous syntax, and natural language obsession—we develop a comprehensive law smell taxonomy. This taxonomy classifies law smells by when they can be detected (static vs. dynamic), which aspects of law they relate to (e.g., structure or grammar), and how they can be discovered (local vs. non-local). We introduce text-based and graph-based methods to identify instances of law smells, confirming their utility in practice using the United States Code as a test case. Our work demonstrates how ideas from software engineering can be leveraged to assess and improve the quality of *legal* code, thus drawing attention to an underappreciated area in the intersection of law and computer science.

Keywords: Refactoring, Software Engineering, Law, Natural Language Processing, Network Analysis

1 Introduction

In modern societies, law is one of the main tools to regulate human activities. These activities are constantly changing, and law co-evolves with them. In the past decades, human activities have become increasingly differentiated and intertwined, e.g., in developments described as *globalization* or *digitization*. Consequently, legal rules, too, have grown more complex, and statutes and regulations have increased in volume, interconnectivity, and hierarchical structure [2, 5].

A similar trend can be observed in software engineering, albeit on a much shorter time scale. The global codebase has

grown exponentially in recent years [12], with GitHub as the largest source code host alone accounting for 250 million repositories between 2008 and 2020.¹ Over the course of its growth, this codebase has become increasingly interconnected when viewed through the lens of network analysis [6]. Naturally, software engineers have turned to technology to keep track of this development and manage code interdependencies. While the challenges for law and software engineering and the constraints within which these challenges must be addressed are not identical, both domains share three important characteristics: *Materially*, their subject matters, legal rules on the one hand and code fragments on the other hand, contain commands intended to control (human or machine) behavior in order to achieve specific outcomes. *Procedurally*, output creation in both law and software engineering is distributed in time and space, and thus, both domains are subject to the challenges of dynamic multi-agent systems. *Methodologically*, lawyers and software engineers alike use abstractions to accommodate problems that are not fully known in the present.

The similarities between the law and software engineering domains suggest possibilities for knowledge transfer between them. In this paper, we explore one such possibility for the software engineering subfield of *refactoring*. Introduced into academic discourse by Opdyke and Johnson [8] and popularized by Becker et al. [1] [Second Edition: 4], refactoring “describes a disciplined technique for restructuring an existing body of code, altering its internal structure without changing its external behavior.”² In software engineering, refactoring is indispensable for ensuring software quality and maintainability, and it is also subject to vivid academic discourse, inspiring detailed analyses of large code bases and dedicated conferences. This has resulted in an actionable understanding of how various variables—e.g., programming

¹According to <https://octoverse.github.com>, 60 million repositories were created in 2020.

²Cf. <https://refactoring.com/>.

Table 1. Overview of distinctions in the law smell taxonomy.

	Categories	Question
Stability	static, dynamic	Can it be detected by looking at the text alone?
Modality	data, structure, grammar, relevance	What aspect of the law does it relate to?
Locality	local, non-local	Can it be detected by looking at a single element of law?

Table 2. Categories used in a law smell profile.

#	Category	Question
1	Description	What is it?
2	Problem	Why is it problematic?
3	Detection	How do we detect it?
4	Mitigation	If we decide that it needs mitigation, how can we mitigate it?
5	Example	For illustration, how does it manifest in practice?

language, team size, project size, or commit size [9], repetition and duplicated code [7], or component size and open source model [10]—impact code quality.

In our current work [3], we demonstrate how concepts from refactoring can be used in law, focusing on the example of *code smells*. At a high level, a code smell is a characteristic of (a part of) the source code that may indicate a deeper problem in its design and implementation, highlighting a need for refactoring [11]. We port this idea to the legal domain, introducing the concept of *law smells*. Law smells constitute the first step towards both (semi-)automatically *detecting* problematic parts of existing codifications and (semi-)automatically *improving* codification quality. They also pave the path towards reproducible, *quantitative* quality measures for collections of legal rules, which allow us to assess the overall *law climate* and facilitate automatic code review and quality assurance.

2 Contribution

Our first and primary contribution is a comprehensive law smell taxonomy, depicted in Figure 1, which we develop based on five intuitive example law smells. This taxonomy, as summarized in Table 1, distinguishes law smells by their *stability*, *modality*, and *locality*, and we profile each law smell using the questions detailed in Table 2.

As our second contribution, we introduce methods for law smell detection. Since legal documents contain *text* with structure that can be represented as a *graph*, our tools fall into two categories: *text-based* and *graph-based*. Text-based tools comprise law-specific adaptations of methods from natural language processing and sequence mining (where the text is understood as a sequence of tokens or characters), while graph-based tools encompass law-specific adaptations of graph algorithms and methods from legal network analysis.

As our third contribution, we apply our law smell detection toolkit to 22 years of codified statutory legislation in the United States Code (1998–2019), confirming the utility of our methods in practice. For example, as shown in Figure 2, we find that *data as text* is ubiquitous and increasingly frequent in the United States Code. This is a sign of *natural language obsession*, and it could be mitigated by introducing typed entities in a data layer augmenting the text.

Overall, we demonstrate how ideas from software engineering can be applied in law, and create numerous opportunities for further research on *defining*, *detecting*, and *deodorizing* law smells. Thus, we hope to contribute a building block for the road towards truly computational law.

3 Outlook

Our current work calls for extensions in three directions. First, we focus on law smell *detection*, but our ultimate goal is law smell *deodorization*. To this end, we need both a better understanding of the landscape of *legal refactorings* and better tools to perform those refactorings identified as applicable—in the best case, *legal integrated development environments* combined with *legal version control*. Second, in many jurisdictions, there is currently no way for the legislator to change the language or structure of the law without creating interpretive uncertainty concerning its content—i.e., refactoring in the classical software engineering sense is hardly possible, and researchers could help legislators change this situation. Third, focusing on an immediately implementable framework, we have deliberately skirted the questions *how* and *to which extent* law can be made computable, which we consider an important research direction. Quests to answer this question might eventually lead us to adapt principles from object-oriented programming to the legal domain, yielding many new law smells and corresponding refactorings.

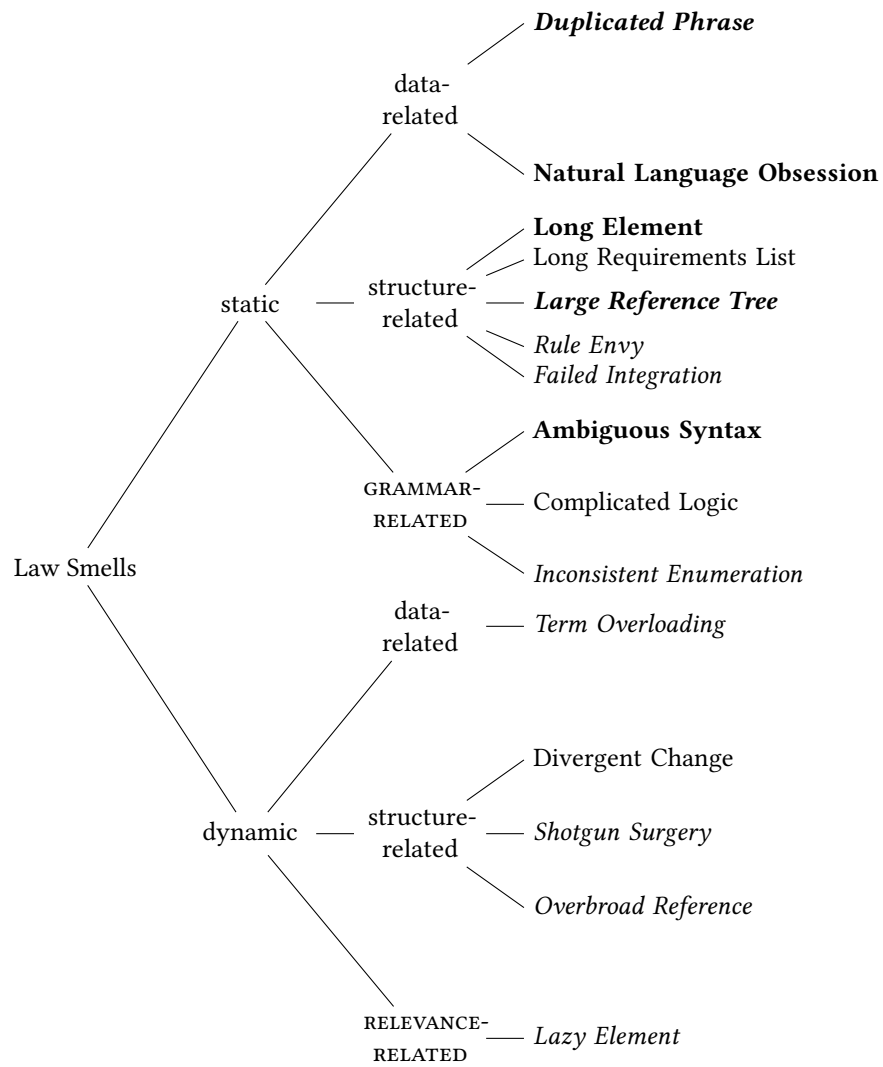


Figure 1. Law smell taxonomy derived in our work. Our example smells are printed in bold, and primarily non-local smells are typeset in italics. The names of subcategories that differ across static and dynamic smells are emphasized using small caps.

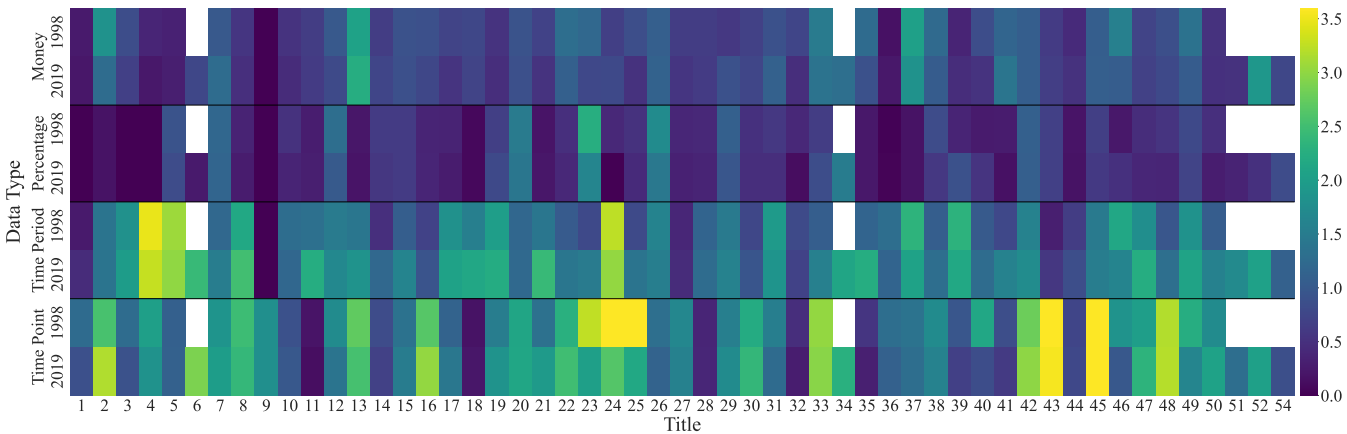


Figure 2. Relative prevalence (instances per 1 000 tokens) of natural language obsession concerning data of the types *money*, *percentage*, *time period*, and *time point*, for each Title in the United States Code in 1998 and 2019.

References

- [1] Paul Becker, Martin Fowler, Kent Beck, John Brant, William Opdyke, and Don Roberts. 1999. *Refactoring: Improving the Design of Existing Code*.
- [2] Corinna Coupette, Janis Beckedorf, Dirk Hartung, Michael Bommarito, and Daniel Martin Katz. 2021. Measuring Law Over Time: A Network Analytical Framework with an Application to Statutes and Regulations in the United States and Germany. *Frontiers in Physics* 9 (2021), 269:1–269:31. <https://doi.org/10.3389/fphy.2021.658463>
- [3] Corinna Coupette, Dirk Hartung, Janis Beckedorf, Maximilian Böther, and Daniel Martin Katz. 2021. Law Smells: Defining and Detecting Problematic Patterns in Legal Drafting. arXiv:2110.11984 [cs.IR]
- [4] Martin Fowler. 2018. *Refactoring: Improving the design of existing code*.
- [5] Daniel Martin Katz, Corinna Coupette, Janis Beckedorf, and Dirk Hartung. 2020. Complex Societies and the Growth of the Law. *Scientific Reports* 10, 1 (2020), 18737:1–18737:14.
- [6] Antonio Lima, Luca Rossi, and Mirco Musolesi. 2014. Coding Together at Scale: GitHub as a Collaborative Social Network. In *Proceedings of the ICWSM*. 295–304.
- [7] Cristina V. Lopes, Petr Maj, Pedro Martins, Vaibhav Saini, Di Yang, Jakub Zitny, Hitesh Sajjani, and Jan Vitek. 2017. DéjàVu: a map of code duplicates on GitHub. *Proceedings of the ACM on Programming Languages* 1, OOPSLA (2017), 1–28. <https://doi.org/10.1145/3133908>
- [8] William F. Opdyke and Ralph E. Johnson. 1990. Refactoring: An aid in designing application frameworks and evolving object-oriented systems. In *Proceedings of the Symposium on Object-Oriented Programming Emphasizing Practical Applications (SOOPPA)*.
- [9] Baishakhi Ray, Daryl Posnett, Premkumar Devanbu, and Vladimir Filkov. 2017. A large-scale study of programming languages and code quality in GitHub. *Commun. ACM* 60, 10 (2017), 91–100.
- [10] Ioannis Stamelos, Lefteris Angelis, Apostolos Oikonomou, and Georgios L. Bleris. 2002. Code quality analysis in open source software development. *Information Systems Journal* 12, 1 (2002), 43–60.
- [11] Michele Tufano, Fabio Palomba, Gabriele Bavota, Rocco Oliveto, Massimiliano Di Penta, Andrea De Lucia, and Denys Poshyvanyk. 2015. When and Why Your Code Starts to Smell Bad. In *Proceedings of the International Conference on Software Engineering (ICSE)*. 403–414.
- [12] Yue Yu, Gang Yin, Huaimin Wang, and Tao Wang. 2014. Exploring the patterns of social behavior in GitHub. In *Proceedings of the International Workshop on Crowd-based Software Development Methods and Technologies (CrowdSoft)*. 31–36. <https://doi.org/10.1145/2666539.2666571>